

# Deep Learning — Week 6 Notes

Autoencoders · Link to PCA · Regularization · Denoising · Sparse · Contractive Autoencoders

NPTEL / IIT Ropar · CS7015 Lecture 7 · Exam-focused revision sheet

## 1 · What is an Autoencoder?

### Encoder → latent code → decoder

An autoencoder is a network trained to **reconstruct its own input**. The **encoder** compresses input  $x$  into a hidden code  $h$ ; the **decoder** rebuilds  $\hat{x}$  from  $h$ . Training minimizes the **reconstruction loss** between  $x$  and  $\hat{x}$ .

$$h = \text{encoder}(x) \cdot \hat{x} = \text{decoder}(h) \cdot L = \sum_i (x_i - \hat{x}_i)^2$$

- **UNDERCOMPLETE**  $\dim(h) < \dim(x)$ : forces compression, learns useful structure.
- **OVERCOMPLETE**  $\dim(h) \geq \dim(x)$ : can just copy input → overfitting risk, needs regularization.
- The **encoder layer creates the latent representation**; the objective is to **minimize reconstruction error** (no external labels).

**Case Study 1** — encoder  $z = w \cdot x$ , decoder reconstructs  $\hat{x}$  from  $z$ :  
latent  $z = 3$  · reconstructed vector  $\hat{x} = (3, 3)^T$  · reconstruction loss = 2.

## 2 · Link Between PCA and Autoencoders

### Linear autoencoder = PCA

If an autoencoder uses a **linear** encoder/decoder and **squared-error** loss, the encoder learns the **same subspace as PCA**. The encoder weight matrix corresponds to the top eigenvectors of the covariance matrix.

- Undercomplete linear AE with  $k$  hidden units  $\approx$  projecting onto the top- $k$  principal components.
- The model is **undercomplete because the latent dimension is smaller than the input dimension**.
- Nonlinear autoencoders can capture structure PCA cannot (e.g. hidden units act like edge detectors).

**Case Study 2** — document features → compact code:  
input dimension = 5 · latent dimension = 2 · undercomplete (latent < input) · output dimension needed to reconstruct the original = 5.

## 3 · Denoising Autoencoders

### Corrupt the input, reconstruct the clean version

A denoising AE deliberately **corrupts** the input (drops or perturbs values) and trains the network to output the **clean** original. The loss is always measured against the clean target, so the model learns robust, stable structure.

encoder sees corrupted  $\tilde{x}$  → decoder outputs  $\hat{x}$  → loss compares  $\hat{x}$  against the CLEAN  $x$

**Case Study 3** — clean reading  $x = (5, 1)^T$ , corrupted input  $\tilde{x} = (5, 0)^T$ , output  $\hat{x} = (4, 1)^T$ :  
vector given to the encoder =  **$(5, 0)^T$**  (corrupted) · clean reconstruction target =  **$(5, 1)^T$**  · loss =  $(5-4)^2 + (1-1)^2 = 1$  · purpose = **train the model to recover clean structure**; the model **maps noisy input toward a clean target**.

## 4 · Sparse & Contractive Autoencoders (Regularization)

### Sparse AE

Adds a **sparsity penalty** so only a **small number of hidden units are active** for any input (average activation pushed toward a small  $\rho$ ). Encourages each unit to specialize.

### Contractive AE

Adds a penalty on the encoder's sensitivity, making the representation **less sensitive to small input changes** — encourages stability.

### Total regularized objective

$$L_{\text{total}} = L_{\text{reconstruction}} + \lambda \cdot (\text{regularization penalty})$$

**Case Study 4** —  $L_{\text{rec}} = 0.8$ , sparsity penalty = 0.3,  $\lambda = 2$ :

sparsity penalty contribution =  $\lambda \times 0.3 = 2 \times 0.3 = \mathbf{0.6}$  ·  $L_{\text{total}} = 0.8 + 0.6 = \mathbf{1.4}$ .

Sparse  $\Rightarrow$  **only a small number of hidden units active** · contractive goal  $\Rightarrow$  **less sensitive to small input changes** · penalties are added to **control the learned representation beyond plain reconstruction**.

### Exam quick-fire (all 20 answers)

- CS1:  $z=3$ ;  $\hat{x}=(3,3)^T$ ; loss=2; encoder layer creates latent; objective = minimize reconstruction error.
- CS2: input dim 5; latent dim 2; undercomplete (latent<input); output dim 5.
- CS3: corrupted input  $(5,0)^T$ ; clean target  $(5,1)^T$ ; loss 1; purpose = recover clean structure; maps noisy  $\rightarrow$  clean.
- CS4: sparsity contribution 0.6;  $L_{\text{total}}=1.4$ ; sparse = few units active; contractive = less input-sensitive; penalties control the representation.